Practical 1: Fibonacci Series Using Recursive and Non-Recursive Methods

Problem Statement: Write a program non-recursive and recursive program to calculate Fibonacci numbers and analyze their time and space complexity.

1. Java Program

Filename: `fibonacci.java`

None

```
import java.util.Scanner;

public class fibonacci {

    // Recursive method
    static int fibonacciRecursive(int n) {

        if (n <= 1) {
            return n;
        }

        return fibonacciRecursive(n - 1) + fibonacciRecursive(n -
2);
    }

    // Non-recursive (Iterative) method
    static void fibonacciIterative(int n) {

        int a = 0, b = 1, c;

        System.out.println("Non-Recursive Fibonacci:");

        for (int i = 0; i < n; i++) {

            System.out.print(a + " ");

            c = a + b;
```

```
            a = b;
            b = c;
        }
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of terms: ");
        int n = sc.nextInt();

        if (n <= 0) {
            System.out.println("Enter a positive number.");
            sc.close();
            return;
        }

        // Calling iterative method
        fibonacciIterative(n);

        // Calling recursive method
        System.out.println("\n\nRecursive Fibonacci:");

        for (int i = 0; i < n; i++) {
            System.out.print(fibonacciRecursive(i) + " ");
        }

        sc.close();
    }
}
```

2. Sample Output

Input:

```
None
Enter number of terms: 5
```

Output:

```
None
Non-Recursive Fibonacci:
0 1 1 2 3

Recursive Fibonacci:
0 1 1 2 3
```

3. Line-by-line explanation

A. Import and class

```
None
import java.util.Scanner;
```

- Imports the Scanner class.
- Scanner is used to accept input from the user.

```
None
public class fibonacci {
```

- Defines a class named `fibonacci`.
- In this program, the filename is also `fibonacci.java`.

B. Recursive method

```
None
static int fibonacciRecursive(int n)
```

- `static`: Method can be called without creating an object.
- `int`: Method returns an integer.
- `n`: Position of the Fibonacci number we want to calculate.

```
None
if (n <= 1) {
    return n;
```

```
}
```

- This is the base condition.
- If n is 0, return 0.
- If n is 1, return 1.
- It stops recursion.

```
None
return fibonacciRecursive(n - 1) + fibonacciRecursive(n - 2);
```

- This is the recursive call.
- The method calls itself with `n-1` and `n-2`.
- It adds both returned values.

For example, to calculate F(4):

$$F(4) = F(3) + F(2)$$

$$= 2 + 1 = 3$$

C. Non-recursive method

```
None
static void fibonacciIterative(int n)
```

- Defines an iterative method.
- `void` means it does not return any value.
- It directly prints the Fibonacci series.

```
None
int a = 0, b = 1, c;
```

- `a` stores the first number.
- `b` stores the second number.
- `c` stores the sum of the previous two numbers.

```
None
for (int i = 0; i < n; i++)
```

- Loop executes n times.
- It prints n Fibonacci terms.

```
None
System.out.print(a + " ");
```

- Prints the current Fibonacci number.

```
None
c = a + b;
a = b;
b = c;
```

These three statements update the numbers.

For example:

a	b	c = a + b
0	1	1
1	1	2
1	2	3
2	3	5

D. Main method

```
None
public static void main(String[] args)
```

- Main is the starting point of Java program execution.
- The JVM starts executing the program from this method.

```
None
Scanner sc = new Scanner(System.in);
```

- Creates a Scanner object to take keyboard input.

```
None
int n = sc.nextInt();
```

- Reads the number of terms entered by the user.

```
None
if (n <= 0)
```

- Checks whether the entered number is positive.
- If not, it displays an error message and stops the program.

```
None
fibonacciIterative(n);
```

- Calls the non-recursive method.

```
None
for (int i = 0; i < n; i++) {
    System.out.print(fibonacciRecursive(i) + " ");
}
```

- Calls the recursive method for positions 0 to n-1.
- Prints each calculated Fibonacci number.

```
None
sc.close();
```

- Closes the Scanner object.

4. Important concepts and theory

What is the Fibonacci series?

A Fibonacci series is a sequence in which every number is the sum of the previous two numbers.

Example:

`0, 1, 1, 2, 3, 5, 8, 13, 21...`

Mathematical formula:

$$F(n) = F(n-1) + F(n-2)$$

With initial values:

$$F(0) = 0, F(1) = 1$$

What is recursion?

Recursion is a programming technique in which a function calls itself to solve a problem.

Example: Our `fibonacciRecursive()` method calls itself to calculate the previous two Fibonacci numbers.

What is iteration?

Iteration means repeating a set of statements using loops such as `for` or `while`.

Example: Our `fibonacciIterative()` method uses a `for` loop to generate the series.

What is a base condition?

A base condition is the condition that stops recursive calls.

In our program:

```
None
if (n <= 1) {
    return n;
}
```

Without a proper base condition, recursion may continue indefinitely until a stack overflow occurs.

Difference between recursion and iteration

Recursion	Iteration
Function calls itself.	Uses loops.
Uses the call stack.	Does not require recursive call stack.
Usually requires more memory.	Usually requires less memory.
Can be slower due to function calls.	Generally faster for this problem.

5. Time and space complexity analysis

This is an important part of your practical because the problem specifically asks you to analyze complexity.

Approach	Time Complexity	Auxiliary Space Complexity
Non-recursive (Iterative)	O(n)	O(1)
Recursive	$O(2^n)$	O(n)

Non-recursive approach:

- The loop runs n times.
- Each iteration performs a constant amount of work.
- Therefore, time complexity is O(n).
- Only three variables (a, b, c) are used, so space complexity is O(1).

Recursive approach:

- Each function call makes two more recursive calls, except at the base cases.

- This creates an expanding recursion tree.
- Therefore, time complexity is $O(2^n)$ (a commonly used upper bound; more tightly, $\Theta(\varphi^n)$).
- The maximum recursion depth is n, so space complexity is O(n).

Note: These complexities describe calculating the nth Fibonacci number. Since our program prints all n terms, the iterative method remains O(n), and the recursive method remains $O(2^n)$ overall.

6. Dry run for input n = 5

The program calculates five terms: `0, 1, 1, 2, 3`.

Iterative approach:

Iteration	Printed value (a)	Updated a	Updated b
1	0	1	1
2	1	1	2
3	1	2	3
4	2	3	5
5	3	5	8

Recursive approach:

For example, when calculating F(4):

```
None
F(4)
 ├── F(3)
 │    ├── F(2)
 │    │    ├── F(1) = 1
 │    │    └── F(0) = 0
```

```
|     └── F(1) = 1
└── F(2)
      ├── F(1) = 1
      └── F(0) = 0
```

Therefore:

- F(0) = 0
- F(1) = 1
- F(2) = 1
- F(3) = 2
- F(4) = 3

7. Viva questions and answers

Q1. What is the Fibonacci series?

A sequence where each number is the sum of the previous two numbers.

Q2. What is the initial value of Fibonacci series?

0 and 1.

Q3. What is recursion?

A function calling itself to solve a problem.

Q4. What is iteration?

Repeating statements using loops.

Q5. What is the base condition in this program?

```
if (n <= 1) return n;
```

Q6. Which approach is faster?

The iterative approach because it takes O(n) time, while the naive recursive approach takes exponential time.

Q7. What is the time complexity of iterative Fibonacci?

O(n).

Q8. What is the time complexity of recursive Fibonacci?

$O(2^n)$, commonly stated as an upper bound.

Q9. What is the space complexity of iterative Fibonacci?

O(1).

Q10. What is the space complexity of recursive Fibonacci?

O(n), because of the recursive call stack.

Q11. What happens if the base condition is removed?

Recursive calls will not stop correctly and may cause a stack overflow.

Q12. Can recursion be optimized?

Yes. Memoization or dynamic programming can reduce Fibonacci calculation to O(n) time.